

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-211БВ-24

Студент: Отмахов Н.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 13.10.25

Москва, 2025

Постановка задачи

Вариант 3.

Группа вариантов 1: Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа, на последующие, а результат выводит в файл. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип `int`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int pipe(int *fd)`; – создает канал между двумя файловыми дескрипторами. Возвращает -1, если возникла ошибка при создании. Заполняет массив `fd`.
`fd[0]` – файловый дескриптор для чтения
`fd[1]` – файловый дескриптор для записи
- `ssize_t readlink(char *path, char* buf, ssize_t bufsiz)`; - считывает содержание символической ссылки и записывает в `buf`.
- `int write(int fd, const void *buf, size_t count)`; – записывает данные из буфера по файловому дескриптору.
- `int read(int fd, void* buf, size_t count)`; - читает данные из файла по файловому дескриптору и записывает в `buf`.
- `int execv(const char *path, char *const argv[])`; - заменяет текущий процесс новым процессом, загружая и выполняя указанную программу
- `pid_t wait(int *wstatus)`; - ожидает завершения любого из дочерних процессов, возвращает информацию о его завершении

Я реализовал межпроцессорное взаимодействие с помощью системных вызовов. Есть родительский процесс, который порождает один дочерний. Он делит первое полученное число на последующие и завершается, если было встречено деление на ноль. Общаются между собой процессы с помощью канала, созданным функцией `pipe`. Пользователь общается только с родительским процессом.

Код программы

client.c

```
#include <stdint.h>
```

```

#include <stdio.h>

#include <stdlib.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>


static char SERVER_PROGRAM_NAME[] = "server";


int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        char msg[1024];

        uint32_t len = snprintf(msg, sizeof(msg) - 1, "usage: %s filename\n", argv[0]);
        write(STDERR_FILENO, msg, len);
        exit(EXIT_FAILURE);
    }

    char progbath[1024];

    {
        ssize_t len = readlink("/proc/self/exe", progbath, sizeof(progbath) - 1);

        if (len == -1) {
            const char msg[] = "error: failed to read full program path\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        }

        while (progbath[len] != '/') {
            --len;
        }

        progbath[len] = '\0';
    }

    int pipe_client_to_child[2];

```

```

int pipe_child_to_client[2];

if (pipe(pipe_client_to_child) == -1)
{
    const char msg[] = "error: failed to create pipe client to server\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(EXIT_FAILURE);
}

if (pipe(pipe_child_to_client) == -1)
{
    const char msg[] = "error: failed to create pipe server to client\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(EXIT_FAILURE);
}

const pid_t child = fork();

switch (child) {
    case -1:
    {
        const char msg[] = "error: failed to spawn new process\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }
    break;

    case 0:
    {
        close(pipe_client_to_child[1]);
        close(pipe_child_to_client[0]);

        dup2(pipe_client_to_child[0], STDIN_FILENO);
        close(pipe_client_to_child[0]);

        dup2(pipe_child_to_client[1], STDOUT_FILENO);
        close(pipe_child_to_client[1]);
    }
}

```

```

{
    char path[1024];
    snprintf(path, sizeof(path) - 1, "%s/%s", progbath, SERVER_PROGRAM_NAME);

    char *const args[] = {SERVER_PROGRAM_NAME, argv[1], NULL};

    int32_t status = execv(path, args);

    if (status == -1)
    {
        const char msg[] = "error: failed to exec into new executable image\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }
}

break;

default:
{
    {
        pid_t pid = getpid();

        char msg[200];
        const int32_t length = snprintf(msg, sizeof(msg),
            "%d: I'm a parent, my child has PID %d\nUsing: 'num num ... num <newline>'\nTo exit
press Enter on empty line\n",
            pid, child);
        write(STDOUT_FILENO, msg, length);
    }

    close(pipe_client_to_child[0]);
    close(pipe_child_to_client[1]);

    char buf[4096];
    ssize_t bytes;

```

```

        while ((bytes = read(STDIN_FILENO, buf, sizeof(buf)))) {
            if (bytes < 0) {
                const char msg[] = "error: failed to read from stdin\n";
                write(STDERR_FILENO, msg, sizeof(msg));
                exit(EXIT_FAILURE);
            } else if (buf[0] == '\n')
            {
                break;
            }

            write(pipe_client_to_child[1], buf, bytes);

            bytes = read(pipe_child_to_client[0], buf, sizeof(buf));
            write(STDOUT_FILENO, buf, bytes);
        }

        close(pipe_client_to_child[1]);
        close(pipe_child_to_client[0]);

        wait(NULL);
    }
    break;
}
}

```

server.c

```

#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main(int argc, char **argv) {
    if (argc != 2) {

```

```

    const char msg[] = "usage: server output_filename\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int32_t file = open(argv[1], O_WRONLY | O_CREAT | O_APPEND, 0600);
if (file == -1) {
    const char msg[] = "error: failed to open requested file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

char line[4096];

while (1) {
    ssize_t bytes = read(STDIN_FILENO, line, sizeof(line) - 1);
    if (bytes <= 0)
        break;

    line[bytes] = '\0';
    if (line[0] == '\n')
        break;

    int32_t nums[100];
    int32_t count = 0;
    char *token = strtok(line, " \t\n");
    while (token && count < 1024) {
        nums[count++] = atoi(token);
        token = strtok(NULL, " \t\n");
    }

    if (count < 2) {
        const char msg[] = "error: need at least two numbers\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        continue;
    }
}

```

```

int32_t first = nums[0];

char outbuf[1024];

for (uint32_t i = 1; i < count; i++) {
    if (nums[i] == 0) {
        const char msg[] = "error: zero division\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        close(file);
        exit(EXIT_FAILURE);
    }

    int32_t result = first / nums[i];

    int32_t len = snprintf(outbuf, sizeof(outbuf), "%d // %d = %d\n", first, nums[i], result);
    write(file, outbuf, len);
}

const char okmsg[] = "Done!\n";
write(STDOUT_FILENO, okmsg, sizeof(okmsg) - 1);
}

close(file);
return 0;
}

```

Протокол работы программы

```

102281: I'm a parent, my child has PID 102282
Using: 'num num ... num <newline>'
To exit press Enter on empty line
12 3 4 5
Done!
1200 3 4 78 99
Done!
939393 249 886 45
Done!
2442 788 0 878 777
error: zero division

```


Текст в выходном файле:

$12 // 3 = 4$

$12 // 4 = 3$

$12 // 5 = 2$

$1200 // 3 = 400$

$1200 // 4 = 300$

$1200 // 78 = 15$

$1200 // 99 = 12$

$939393 // 249 = 3772$

$939393 // 886 = 1060$

$939393 // 45 = 20875$

$2442 // 788 = 3$

Вывод

В ходе данной лабораторной работы я научился управлять процессами в ОС. Также обеспечил обмен данных между процессами посредством каналов. Основной проблемой в ходе выполнения работы было изучение функций, которые реализуют взаимодействие между процессами.